WEB DEVELOPMENT UNLOCKED

Build Wallora from Scratch — A Complete Beginner’s Guide to TypeScript, React, Next.js, CSS, SQL & Modern Web Development

With Real Code from the Wallora Project • 2026 Edition

Contents

1. Welcome to Web Development

Before we write a single line of code, let's understand what we're building and why. This chapter sets the stage for everything that follows.

1.1 What Are We Building?

Wallora (Aurava) is a real, production-grade wallpaper marketplace. Users can browse a catalog of free and premium wallpapers, add them to a cart, pay for premium ones using mobile money or cards, and download them instantly. An admin dashboard lets the team manage wallpapers, blog posts, categories, and orders. AI helps auto-tag wallpapers. The whole thing is live on the internet at `auravaw.tech`.

By the end of this book, you won't just understand how Wallora works — you'll be able to build something like it yourself. Every concept is explained from the ground up. No prior experience needed.

1.2 The Languages You'll Learn

Language	What It's For	Where in Wallora
HTML / JSX	Structure of web pages	Every `.tsx` file
CSS	Visual styling, layout, colors	`globals.css`, Tailwind classes
JavaScript	Making pages interactive	Cart logic, animations
TypeScript	JS with type safety	Almost all `.ts/.tsx` files
SQL	Database queries	`supabase/migrations/*.sql`
JSON	Configuration & data	`package.json`, `tsconfig.json`
YAML	Configuration	`pnpm-workspace.yaml`

1.3 The Frameworks & Tools

Tool	Purpose
Next.js 16	Full-stack React framework (pages, APIs, server actions)
React 19	UI library for building components
Tailwind CSS v4	Utility-first CSS framework
Supabase	Database, authentication, file storage
Zod v4	Data validation
Framer Motion	Animations
PesaPal v3	Payment processing
Resend	Email delivery
Cloudinary	Image hosting & optimization
Google Gemini	AI image analysis

1.4 How This Book Works

Each chapter introduces concepts using real code from Wallora. You'll see the exact same patterns the pros use. At the end of most chapters, there's a **hands-on challenge** for you to try.

> **How to read this book**
>
> If you're a complete beginner, read chapters 1–6 in order. If you already know some JavaScript, skim chapters 4–5 and start at chapter 6. Either way, have a code editor ready — the best way to learn is by typing code yourself.

Chapter 1 Key Points

- Wallora is a production wallpaper marketplace built with Next.js, React, TypeScript, and Supabase
- You'll learn HTML, CSS, JavaScript, TypeScript, SQL, JSON, and YAML
- No prior experience needed — everything is explained from scratch
- Real Wallora code is used throughout as examples

2. HTML & JSX — The Structure of the Web

Every website you've ever visited is built on HTML. It's the skeleton of the web. This chapter teaches HTML from scratch, then explores JSX.

2.1 What Is HTML?

HTML stands for **HyperText Markup Language**. It's not a programming language — it's a *markup* language. It tells the browser what to show (headings, paragraphs, images, links) and how to organize them. HTML uses **tags** (also called elements) to mark up content.

```
<!-- This is a comment in HTML -->
<h1>Hello, World!</h1>
<p>This is a paragraph.</p>
<a href="https://auravaw.tech">Visit Aurava</a>
```

2.2 The Structure of an HTML Page

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Welcome!</h1>
    <p>Content goes here.</p>
  </body>
</html>
```

2.3 Common HTML Elements

Tag	Purpose	Example
`<h1>` to `<h6>`	Headings	`<h1>Wallora</h1>`
`<p>`	Paragraph	`<p>Browse our collection ... </p>`
`<a>`	Link	`<a href="/cart">Cart</a>`
`<img>`	Image	`<img src="wp.jpg" alt=" ... ">`
`<div>`	Generic container	`<div class="container"> ... </div>`
`<button>`	Clickable button	`<button>Buy Now</button>`
`<ul>` / `<ol>`	Unordered / ordered list	`<ul><li>Item</li></ul>`
`<input>`	Form input field	`<input type="email">`
`<form>`	Form container	`<form action="/login"> ... </form>`

2.4 Attributes

HTML tags can have **attributes** — extra information that modifies behavior:

```
<a href="https://auravaw.tech" class="btn" target="_blank">Visit</a>
<img src="photo.jpg" alt="Description" width="800" height="600" />
```

2.5 What Is JSX?

JSX is a syntax extension for JavaScript that looks like HTML but lives inside JavaScript/TypeScript files. Wallora uses JSX (in `.tsx` files) to describe what the UI should look like.

```
export function WallpaperCard({ w }: { w: CardWallpaper }) {
  return (
    <div className="group relative overflow-hidden rounded-2xl bg-surface">
      <div className="relative aspect-[9/16] overflow-hidden">
        <Image
          src={previewUrl(w, { width: 600 })}
          alt={w.title}
          width={600}
          height={1067}
        />
      </div>
    </div>
  );
}
```

2.6 HTML vs JSX: Key Differences

HTML	JSX
`class="name"`	`className="name"`
`for="input"`	`htmlFor="input"`
`<img>` (no close)	`<img />` (must close)
Attributes are strings	Attributes can be JS: `src={variable}`

2.7 Self-Closing Tags

In JSX, every tag must be closed. Tags without children use self-closing syntax:

```
<!-- HTML -->
<input type="email" name="email">

{/* JSX */}
<input type="email" name="email" />
```

2.8 JSX Expressions with Curly Braces

You can embed any JavaScript expression inside **curly braces** `{ }`:

```
<div className="flex items-center gap-3">
  <span className="text-2xl font-bold">
    {formatPrice(w.priceCents)}
  </span>
  <button onClick={() => cart.add(w)}>Add to Cart</button>
</div>
```

2.9 Conditional Rendering

```
{/* Show premium badge only for premium */}
{w.isPremium && <Badge variant="accent">Premium</Badge>}

{/* If/else with ternary */}
{cart.has(w.id)
  ? <button onClick={() => cart.remove(w.id)}>Remove</button>
  : <button onClick={() => cart.add(w)}>Add to Cart</button>
}
```

2.10 Lists and Keys

```
<div className="masonry">
  {wallpapers.map((w) => (
    <WallpaperCard key={w.id} w={w} />
  ))}
</div>
```

Why keys matter

React uses `key` to track list items when they change. Always use stable, unique IDs (like `w.id`), never array indexes.

2.11 Semantic HTML

Use tags by their meaning, not appearance. This helps accessibility and SEO.

Instead of	Use	Meaning
`<div id="header">`	`<header>`	Page header
`<div id="nav">`	`<nav>`	Navigation
`<div class="article">`	`<article>`	Self-contained content
`<div id="footer">`	`<footer>`	Page footer

2.12 Exercise: Your First JSX

Write JSX that creates a simple wallpaper card with a title, description, and button:

```
const title = "Mountain Sunset";
const price = 299;

function Card() {
  return (
    <div className="card">
      <h2>{title}</h2>
      <p>Price: ${price}</p>
      <button>Buy Now</button>
    </div>
  );
}
```

Chapter 2 Key Points

- HTML provides structure using tags
- JSX is HTML-like syntax inside JavaScript/TypeScript files
- Key difference: `className` instead of `class`, all tags must close
- Use `{ }` to embed JavaScript expressions in JSX
- Use `&&` and `? :` for conditional rendering
- Use `.map()` with unique `key` props to render lists

3. CSS & Tailwind — Making Things Beautiful

HTML gives structure. CSS gives style — colors, layouts, spacing, fonts. In Wallora, we use Tailwind CSS, a modern approach that applies styles directly in HTML using utility classes.

3.1 What Is CSS?

CSS (Cascading Style Sheets) describes how HTML elements should look:

```
h1 {
  color: blue;
  font-size: 32px;
}
```

3.2 The CSS Box Model

Every HTML element is a rectangular box. From inside out: Content → Padding → Border → Margin.

```
div {
  width: 300px;
  padding: 20px;    /* Space inside the border */
  border: 2px solid black;
  margin: 10px;     /* Space outside */
}
```

3.3 Colors in Modern CSS (OKLCH)

Wallora uses the modern **OKLCH** color space for richer, more consistent colors across dark/light modes:

```
/* Traditional */
color: #ff0000;           /* Hex */
color: rgb(255, 0, 0);    /* RGB */

/* Modern OKLCH (what Wallora uses) */
color: oklch(40% 0.15 30);  /* Lightness, Chroma, Hue */

/* Wallora's root variables */
:root {
  --background: oklch(0.13 0.02 285);
  --foreground: oklch(0.95 0.01 285);
  --accent: oklch(0.72 0.16 70);
  --surface: oklch(0.18 0.02 285);
}
```

3.4 Layout: Flexbox

Flexbox arranges items in a row or column:

```
/* CSS */
.nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
}

/* Tailwind */
<div className="flex items-center justify-between gap-4">...</div>
```

3.5 The Masonry Layout

Wallora uses pure CSS masonry (like Pinterest) for wallpaper grids:

```
/* src/app/globals.css */
.masonry {
  column-count: 1;
  column-gap: 1rem;
}
@media (min-width: 640px) { .masonry { column-count: 2; } }
@media (min-width: 1024px) { .masonry { column-count: 3; } }
@media (min-width: 1536px) { .masonry { column-count: 4; } }

.masonry > * {
  break-inside: avoid;
  margin-bottom: 1rem;
  content-visibility: auto;
}
```

3.6 Responsive Design

Websites must work on phones, tablets, and desktops:

```
/* CSS */
.container { padding: 1rem; }
@media (min-width: 768px) { .container { padding: 2rem; } }

/* Tailwind: p-4 (all), md:p-8 (768px+), lg:p-16 (1024px+) */
<div className="p-4 md:p-8 lg:p-16">...</div>
```

3.7 Tailwind CSS: Utility-First

Tailwind provides small utility classes that you compose directly in HTML:

What You Want	Tailwind Class
Padding	`p-4`
Margin top	`mt-2`
Flexbox	`flex`
Text color from theme	`text-muted`
Background from theme	`bg-surface`
Rounded corners	`rounded-2xl`
Hover effect	`hover:bg-accent`
Responsive	`md:flex`
Dark mode	`dark:bg-black`

3.8 Tailwind Theme Configuration

Tailwind v4 lets you define a theme inline:

```
/* Wallora's globals.css */
@theme inline {
  --color-background: var(--background);
  --color-surface: var(--surface);
  --color-accent: var(--accent);
  --color-muted: var(--muted);
  --radius-card: 14px;
}
```

3.9 The Glass Effect

Wallora's navbar uses glassmorphism:

```
.glass {
  background: color-mix(in oklch, var(--background) 72%, transparent);
  backdrop-filter: blur(14px) saturate(150%);
}
```

3.10 Dark & Light Mode

```
/* Dark-first design */
:root { /* dark defaults */ }
.light { /* light overrides */ }
```

3.11 Skeleton Loading

```
.skeleton::after {
  content: "";
  position: absolute; inset: 0;
  transform: translateX(-100%);
  background: linear-gradient(90deg, transparent,
    color-mix(in oklch, var(--foreground) 7%, transparent), transparent);
  animation: shimmer 1.6s ease-in-out infinite;
}
@keyframes shimmer {
  to { transform: translateX(100%); }
}
```

3.12 Reduced Motion

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.001ms !important;
    transition-duration: 0.001ms !important;
  }
}
```

3.13 The `cn()` Utility

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

3.14 Exercise: Style a Card with Tailwind

```
<div className="group relative overflow-hidden rounded-2xl bg-surface
            shadow-lg transition-all duration-300
            hover:shadow-xl hover:-translate-y-1">
  <div className="relative aspect-[9/16] overflow-hidden">
    <img
      src="https://placehold.co/600×1067"
      alt="Wallpaper"
      className="h-full w-full object-cover transition-all
                duration-500 group-hover:scale-105"
    />
  </div>
  <div className="p-4">
    <h3 className="font-semibold">Mountain View</h3>
    <p className="mt-1 text-sm text-muted">$2.99</p>
  </div>
</div>
```

Chapter 3 Key Points

- CSS controls visual presentation: colors, layout, spacing, fonts
- The box model: content → padding → border → margin
- Tailwind CSS uses utility classes instead of custom CSS
- OKLCH provides better colors than hex or RGB
- Media queries and Tailwind prefixes make sites responsive
- Always respect `prefers-reduced-motion`

4. JavaScript — The Brain of the Web

HTML and CSS build structure and appearance. JavaScript brings pages to life — handling clicks, fetching data, updating content, and more. Wallora's entire frontend and backend is JavaScript.

4.1 Variables: Storing Data

```
const name = "Aurava";    // Can't be reassigned (use by default)
let count = 0;            // Can be reassigned
var oldWay = "avoid";     // Old style, don't use

name = "Wallora";  // ERROR! const can't change
count = 5;         // OK! let can change
```

4.2 Data Types

Type	Example	Description
`string`	`"hello"`	Text
`number`	`42` , `3.14`	Integers and decimals
`boolean`	`true` , `false`	Yes/no
`null`	`null`	Intentional nothing
`undefined`	`undefined`	Not assigned yet
`object`	`{ name: "Wallora" }`	Key-value pairs
`array`	`[1, 2, 3]`	Ordered lists

```
const wallpaper = {
  title: "Mountain",
  priceCents: 299,
  isPremium: true
};

const tags = ["nature", "sunset"];
```

4.3 Operators

```
// Arithmetic
const sum = 5 + 3;              // 8

// Comparison (always ===, never ==)
const isEqual = 5 === "5";      // false (different types)

// Logical
const and = true && false;      // false
const or = true || false;       // true

// Nullish coalescing (ES2020)
const displayName = name ?? "Guest";  // Only if null/undefined

// Optional chaining (safe access)
const city = user?.address?.city;  // No crash if null
```

4.4 Functions

```
// Arrow function (modern, used throughout Wallora)
const add = (a, b) => a + b;

// Real example from Wallora's utils.ts
export function formatPrice(cents: number, currency = "USD", locale = "en-US"): string {
  return new Intl.NumberFormat(locale, {
    style: "currency",
    currency,
  }).format(cents / 100);
}
// formatPrice(299) → "$2.99"
```

4.5 Objects & Destructuring

```
const wallpaper = {
  id: "wp_001",
  title: "Mountain Sunset",
  priceCents: 299,
  tags: ["nature", "sunset"]
};

// Destructuring (very common in Wallora!)
const { title, priceCents } = wallpaper;

// Spread operator
const updated = { ...wallpaper, priceCents: 499 };
```

4.6 Essential Array Methods

```
const prices = [299, 499, 0, 999];

// .map() — transform every element (most common in React/JSX)
const withTax = prices.map(p => p * 1.16);

// .filter() — keep only matching
const paid = prices.filter(p => p > 0);

// .find() — first match
const free = prices.find(p => p === 0);

// .reduce() — accumulate
const total = prices.reduce((sum, p) => sum + p, 0);  // 1797

// .some() / .every()
prices.some(p => p > 500);   // true
prices.every(p => p >= 0);  // true
```

4.7 Async/Await & Promises

JavaScript handles async tasks (database, API calls) without blocking:

```
// Real Wallora pattern: parallel data fetching
const [featured, week, popular, fresh, live, categories] = await Promise.all([
  getFeaturedForDisplay("day"),
  getFeaturedForDisplay("week"),
  listWallpapers({ sort: "popular", limit: 12 }),
  listWallpapers({ sort: "newest", limit: 8 }),
  listWallpapers({ kind: "live", limit: 8 }),
  listCategories(),
]);
```

All six fetches run in parallel. This makes the homepage load much faster than fetching one at a time.

4.8 Error Handling with Try/Catch

```
try {
  const data = await fetchSomeData();
} catch (error) {
  console.error("Failed:", error);
  showErrorMessage("Something went wrong");
}
```

4.9 Modules: Import & Export

```
// utils.js
export function formatPrice(cents) { ... }
export const TAX_RATE = 0.16;

// cart.js
import { formatPrice, TAX_RATE } from "./utils";
```

4.10 Exercise: Process Wallpapers

```
const wallpapers = [
  { title: "Sunset", priceCents: 499, isPremium: true },
  { title: "Forest", priceCents: 0, isPremium: false },
  { title: "Ocean", priceCents: 999, isPremium: true },
];

const discountedPremium = wallpapers
  .filter(w => w.isPremium)
  .map(w => ({ ...w, priceCents: w.priceCents * 0.8 }))
  .sort((a, b) => a.priceCents - b.priceCents);

const total = discountedPremium.reduce((s, w) => s + w.priceCents, 0);
console.log(`Total: $${(total / 100).toFixed(2)}`);
```

Chapter 4 Key Points

- Use `const` by default, `let` when reassignment is needed
- Arrow functions are the modern way to write functions
- Destructuring and spread make working with objects/arrays cleaner
- `.map()`, `.filter()`, `.find()`, `.reduce()` are essential array methods
- Async/await + Promise.all handle parallel async operations
- Always use `===` (not `==`) for comparisons

5. TypeScript — JavaScript with Superpowers

TypeScript is JavaScript with **types**. Types catch errors before your code runs, make your code self-documenting, and give you better autocomplete. Wallora is written entirely in TypeScript.

5.1 Why TypeScript?

TypeScript prevents errors at compile time (before code runs). It adds a type-checking phase that catches bugs like passing `undefined` where an object is expected.

5.2 Basic Types

```
const name: string = "Wallora";
const price: number = 299;
const isPremium: boolean = true;
const tags: string[] = ["nature", "sunset"];

// TypeScript can infer types automatically:
const inferred = "hello";  // TypeScript knows it's a string
```

5.3 Interfaces & Type Aliases

```
// Interface (preferred for object shapes)
interface Wallpaper {
  id: string;
  title: string;
  priceCents: number;
  isPremium: boolean;
  tags: string[];
  readonly slug: string;
  createdAt?: string;  // ? means optional
}

// Type alias (for unions and simpler types)
type DeviceType = "desktop" | "phone" | "tablet";
type AgeRating = "everyone" | "13+" | "16+" | "18+";
type OrderStatus = "pending" | "paid" | "failed" | "cancelled";

// Using them
function canView(viewer: Viewer, rating: AgeRating): boolean { ... }
```

Real type from Wallora's `src/lib/types.ts`:

```ts
export interface Wallpaper {
  id: string;
  slug: string;
  title: string;
  description: string;
  originalPublicId: string;
  originalStoragePath: string;
  previewPublicId: string;
  kind: "image" | "live";
  videoPublicId: string | null;
  durationSec: number | null;
  categorySlug: string;
  tags: string[];
  device: DeviceType;
  resolution: string;
  width: number;
  height: number;
  ageRating: AgeRating;
  isMature: boolean;
  priceCents: number;
  isPremium: boolean;
  seoTitle: string;
  seoDescription: string;
  isFeatured: boolean;
  holidayTags: string[];
  downloads: number;
  createdAt: string;
}
```

5.4 Union Types (Discriminated Unions)

```ts
type Result<T> =
  | { ok: true; data: T }
  | { ok: false; error: string };

// TypeScript narrows the type based on the 'ok' property
const result: Result<Wallpaper[]> = await getWallpapers();
if (result.ok) {
  console.log(result.data);  // TS knows data is Wallpaper[]
} else {
  console.error(result.error);  // TS knows error is string
}
```

5.5 Generics

```ts
// Reusable function for any type
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}

const firstWallpaper = first<Wallpaper>(wallpapers);
const firstNumber = first([1, 2, 3]);  // inferred as number
```

5.6 Type Utilities

```
type PartialWallpaper = Partial<Wallpaper>;  // All optional
type WallpaperPreview = Pick<Wallpaper, "id"|"title">;  // Just these keys
type NoPrice = Omit<Wallpaper, "priceCents">;  // Exclude key
```

5.7 Zod: Validation + Type Inference

```
import { z } from "zod";

const schema = z.object({
  email: z.string().email(),
  ids: z.array(z.string().min(1)).min(1),
});

// Extract TypeScript type from the schema (single source of truth!)
type CheckoutInput = z.infer<typeof schema>;
// → { email: string; ids: string[] }
```

5.8 Type Narrowing

```
if (profile === null) {
  return;  // TS knows profile is null here
}
console.log(profile.displayName);  // TS knows profile is not null here
```

5.9 Exercise: Type a Checkout Function

```
interface CheckoutInput {
  email: string;
  wallpaperIds: string[];
}

interface CheckoutResult {
  ok: boolean;
  redirectUrl?: string;
  error?: string;
}

async function checkout(input: CheckoutInput): Promise<CheckoutResult> {
  if (!input.email) {
    return { ok: false, error: "Email required" };
  }
  // Process checkout ...
  return { ok: true, redirectUrl: "/checkout/callback" };
}
```

Chapter 5 Key Points

- TypeScript adds static types to JavaScript, catching errors before runtime
- Interfaces define object shapes; type aliases define unions and primitives
- Union types with discriminated patterns make code safer
- Generics (`<T>`) create reusable, type-safe functions
- Utility types like `Partial<T>` , `Pick<T>` transform existing types
- `z.infer<typeof schema>` extracts TypeScript types from Zod schemas

6. React — Building User Interfaces

React lets you build UIs out of **components** — reusable pieces that manage their own state and compose together. Wallora has dozens of components: navbar, wallpaper cards, cart, checkout forms, and more.

6.1 What Is React?

A React component is just a function that returns JSX:

```
function Greeting() {
  return <h1>Hello, World!</h1>;
}

// Usage:
<Greeting />
```

6.2 Components with Props

Props are inputs to components, like function parameters:

```
interface WallpaperCardProps {
  w: CardWallpaper;
  index?: number;
}

// From Wallora's src/components/wallpaper-card.tsx
export function WallpaperCard({ w, index = 0 }: WallpaperCardProps) {
  return (
    <ScrollReveal index={index}>
      <div className="group relative overflow-hidden rounded-2xl bg-surface">
        <div className="relative aspect-[9/16] overflow-hidden">
          <ProtectedImage
            src={previewUrl(w, { width: 600 })}
            alt={w.title}
          />
        </div>
        <div className="absolute inset-x-0 bottom-0 p-4">
          <h3 className="font-semibold text-white">{w.title}</h3>
          {w.isPremium
            ? <span className="text-accent text-sm">{formatPrice(w.priceCents)}</span>
            : <Badge variant="free">Free</Badge>
          }
        </div>
      </div>
    </ScrollReveal>
  );
}
```

6.3 State: Data That Changes

`useState` creates state variables. When state changes, React re-renders the component:

```
"use client";
import { useState } from "react";

export function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
}
```

6.4 The `useEffect` Hook

`useEffect` runs side effects (like localStorage, API calls):

```
useEffect(() => {
  // Load cart from localStorage (once on mount)
  const raw = localStorage.getItem("aurava_cart");
  if (raw) setLines(JSON.parse(raw));
  setHydrated(true);
}, []);  // Empty array = run once

useEffect(() => {
  // Save to localStorage when cart changes
  localStorage.setItem("aurava_cart", JSON.stringify(lines));
}, [lines]);  // Runs when 'lines' changes
```

6.5 Context: Sharing Data Across the Tree

Context lets you share data without passing props through every level:

```tsx
const CartContext = createContext<CartContextValue | null>(null);

export function CartProvider({ children }: { children: ReactNode }) {
  const [lines, setLines] = useState<CartLine[]>([]);

  const add = useCallback((w: CardWallpaper) => {
    setLines(prev => [...prev, {
      wallpaperId: w.id, title: w.title, priceCents: w.priceCents
    }]);
  }, []);

  const remove = useCallback((id: string) => {
    setLines(prev => prev.filter(l => l.wallpaperId !== id));
  }, []);

  const totalCents = useMemo(() =>
    lines.reduce((sum, l) => sum + l.priceCents, 0),
  [lines]);

  return (
    <CartContext.Provider value={{ lines, add, remove, totalCents }}>
      {children}
    </CartContext.Provider>
  );
}

export function useCart(): CartContextValue {
  const ctx = useContext(CartContext);
  if (!ctx) throw new Error("useCart must be used within CartProvider");
  return ctx;
}
```

6.6 Event Handling

```tsx
<button onClick={() => cart.add(w)}>Add to Cart</button>

<input onChange={(e) => setEmail(e.target.value)} value={email} />

<form onSubmit={(e) => {
  e.preventDefault();
  handleSubmit();
}}> ... </form>
```

6.7 Client Components (“use client”)

In Wallora, interactive components must be marked as client components:

```
"use client";

import { motion } from "motion/react";
import { useCart } from "@/components/cart";

export function WallpaperCard({ w }: { w: CardWallpaper }) {
  const cart = useCart();
  return ( /* JSX with interactivity */ );
}
```

6.8 Exercise: Build a Product Card

Create a React component that shows a wallpaper title, price, and an “Add to Cart” button. Track whether it’s been added using `useState`:

```
"use client";
import { useState } from "react";

interface Product {
  id: string;
  title: string;
  priceCents: number;
}

export function ProductCard({ product }: { product: Product }) {
  const [added, setAdded] = useState(false);

  return (
    <div className="rounded-2xl bg-surface p-4">
      <h3>{product.title}</h3>
      <p>{formatPrice(product.priceCents)}</p>
      <button
        onClick={() => setAdded(!added)}
        className="mt-2 rounded-lg bg-accent px-4 py-2 font-medium text-white"
      >
        {added ? "Remove" : "Add to Cart"}
      </button>
    </div>
  );
}
```

Chapter 6 Key Points

- React components are functions that return JSX
- Props pass data into components like function arguments
- `useState` creates state; when it changes, the component re-renders
- `useEffect` runs side effects (localStorage, API calls, subscriptions)
- Context shares data through the component tree without prop drilling
- Client components need `"use client"` directive

7. Next.js — The Full-Stack Framework

Next.js is a React framework that gives you file-based routing, server-side rendering, API routes, and more. Wallora runs on Next.js 16 with the App Router.

7.1 What is Next.js?

Next.js extends React with powerful features:

- **File-based routing** — files in `src/app/` become URLs automatically
- **Server Components** — components that run on the server, sending only HTML to the browser
- **Server Actions** — functions that run on the server, called from the browser
- **API Routes** — backend endpoints in `src/app/api/`
- **Image Optimization** — automatic image resizing and optimization

7.2 File-Based Routing

In Next.js App Router, the file structure maps directly to URLs:

```
src/app/
  page.tsx           →  /
  layout.tsx          →  Layout wrapper for all pages
  wallpapers/
    page.tsx          →  /wallpapers
    [slug]/
      page.tsx        →  /wallpapers/mountain-sunset (dynamic)
  cart/
    page.tsx          →  /cart
  checkout/
    page.tsx          →  /checkout
    callback/
      page.tsx        →  /checkout/callback
  api/
    pesapal/ipn/
      route.ts        →  /api/pesapal/ipn (POST webhook)
```

Wallora's route structure:

File	URL	Purpose
`src/app/page.tsx`	`/`	Homepage
`src/app/wallpapers/page.tsx`	`/wallpapers`	Catalog with filters
`src/app/wallpapers/[slug]/page.tsx`	`/wallpapers/:slug`	Category or wallpaper detail
`src/app/cart/page.tsx`	`/cart`	Shopping cart
`src/app/checkout/page.tsx`	`/checkout`	Payment checkout
`src/app/admin-dash/page.tsx`	`/admin-dash`	Admin overview
`src/app/api/pesapal/ipn/route.ts`	`/api/pesapal/ipn`	Payment webhook

7.3 Layouts

Layouts wrap pages and persist across navigations. Wallora's root layout provides navbar, footer, cart, and currency contexts:

```
// src/app/layout.tsx (simplified)
export default async function RootLayout({ children }: { children: ReactNode }) {
  const [viewer, hdrs] = await Promise.all([getViewer(), headers()]);
  const country = hdrs.get("x-vercel-ip-country") ?? "US";

  return (
    <html lang="en" suppressHydrationWarning>
      <body className="flex min-h-full flex-col">
        <CartProvider>
          <CurrencyProvider defaultCountry={country}>
            <Navbar displayName={viewer.profile?.displayName ?? null}
                    isAdmin={viewer.isAdmin} />
            <main className="flex-1">{children}</main>
            <Footer />
          </CurrencyProvider>
        </CartProvider>
      </body>
    </html>
  );
}
```

7.4 Route Groups

Parentheses in folder names create route groups that don't affect the URL:

```
src/app/
  (auth)/              // Route group for auth pages
    login/page.tsx     // → /login
    signup/page.tsx    // → /signup
  (legal)/             // Route group for legal pages
    terms/page.tsx     // → /terms
    privacy/page.tsx   // → /privacy
    refund/page.tsx    // → /refund
```

7.5 Dynamic Routes

Square brackets `[slug]` create dynamic routes. Wallora's slug page handles both categories AND wallpapers:

```
// src/app/wallpapers/[slug]/page.tsx
export default async function SlugPage({ params }: { params: Params }) {
  const { slug } = await params;
  const category = await repo.getCategory(slug);

  if (category) {
    return <CategoryPage category={category} />;
  }

  const w = await getViewableWallpaper(slug);
  if (!w) notFound();

  return <WallpaperDetailPage wallpaper={w} />;
}
```

7.6 Loading and Error States

Next.js provides built-in loading and error files:

```
// src/app/wallpapers/loading.tsx
export default function Loading() {
  return <div className="masonry">
    {Array.from({ length: 8 }).map((_, i) => (
      <div key={i} className="aspect-[9/16] skeleton rounded-2xl" />
    ))}
  </div>;
}
```

7.7 Route Handlers (API Routes)

API routes are defined in `route.ts` files. They handle HTTP requests:

```
// src/app/api/pesapal/ipn/route.ts — Payment webhook
export async function GET(request: NextRequest) {
  const sp = request.nextUrl.searchParams;
  const trackingId = sp.get("pesapal_transaction_tracking_id");
  const merchantRef = sp.get("pesapal_merchant_reference");

  if (!trackingId || !merchantRef) {
    return NextResponse.json({ error: "Missing params" }, { status: 400 });
  }

  const order = await fulfillByMerchantRef(merchantRef);
  return NextResponse.json(order);
}

export const POST = GET;
```

7.8 Proxy (Middleware)

Next.js 16 renamed `middleware.ts` to `proxy.ts`. Wallora uses it for auth gating:

```
// src/proxy.ts
export async function proxy(request: NextRequest) {
  const response = NextResponse.next({ request });
  // Session refresh + route gating
  return response;
}

export const config = {
  matcher: ["/((?!_next/static|...).*)"],
};
```

7.9 Metadata & SEO

Next.js has a built-in Metadata API for SEO:

```
import { Metadata } from "next";

export const metadata: Metadata = {
  title: "Wallora — Premium 4K & HD Wallpapers",
  description: "Download stunning wallpapers...",
  openGraph: {
    title: "Wallora",
    images: [{ url: "/og-image.png", width: 1200, height: 630 }],
  },
};
```

7.10 Exercise: Create a Page

Create a simple Next.js page at `src/app/gallery/page.tsx` that displays a list of wallpapers:

```tsx
// src/app/gallery/page.tsx
import { Container } from "@/components/ui";

async function getWallpapers() {
  const res = await fetch("https://api.example.com/wallpapers");
  return res.json();
}

export default async function GalleryPage() {
  const wallpapers = await getWallpapers();

  return (
    <Container>
      <h1>Gallery</h1>
      <div className="masonry">
        {wallpapers.map((w: Wallpaper) => (
          <div key={w.id} className="rounded-2xl bg-surface p-4">
            <h3>{w.title}</h3>
          </div>
        ))}
      </div>
    </Container>
  );
}
```

Chapter 7 Key Points

- Next.js App Router uses file-based routing: files in `src/app/` become URLs
- Dynamic routes use square brackets: `[slug]`
- Route groups (parentheses) organize without affecting URLs
- Layouts wrap pages and persist across navigations
- Route handlers (`route.ts`) create API endpoints
- Built-in Metadata API for SEO

8. Server Components & Server Actions

Next.js 16 introduces two revolutionary concepts: Server Components and Server Actions. They let you write backend code directly in your frontend files, making apps faster and simpler.

8.1 Server Components (Default)

In Next.js App Router, **every component is a Server Component by default**. Server Components:

- Run on the server, never on the browser
- Can directly access databases, file systems, and APIs
- Send only HTML to the client (no JavaScript)
- Can `await` async data directly in the component body

```
// This is a Server Component — no “use client” directive
import { repo } from "@/lib/repo";

export default async function WallpapersPage() {
  // Directly await data — no useEffect, no state, no loading flags
  const wallpapers = await repo.listWallpapers({ limit: 20 });

  return (
    <div className="masonry">
      {wallpapers.map(w => (
        <WallpaperCard key={w.id} w={w} />
      ))}
    </div>
  );
}
```

8.2 Client Components (“use client”)

Add `"use client"` at the top of the file when you need interactivity (event handlers, state, effects):

```
"use client";
import { useState } from "react";

export function CartButton() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)>Clicks: {count}</button>;
}
```

8.3 Server Actions (“use server”)

Server Actions are functions that run on the server but can be called from client components. They’re used for form submissions, mutations, and any backend logic:

```
// src/app/checkout/actions.ts
"use server";
import { z } from "zod";

const schema = z.object({
  email: z.string().email(),
  ids: z.array(z.string().min(1)).min(1),
});

export async function beginCheckout(email: string, ids: string[]): Promise<CheckoutResult> {
  // 1. Validate input with Zod
  const parsed = schema.safeParse({ email, ids });
  if (!parsed.success) {
    return { ok: false, error: parsed.error.issues[0]?.message ?? "Invalid input" };
  }

  try {
    // 2. Server logic (runs on server, safe for secrets & DB)
    const viewer = await getViewer();
    const { redirectUrl } = await startCheckout(
      ids.map(id => ({ wallpaperId: id })),
      email,
      viewer.profile?.id ?? null
    );

    return { ok: true, redirectUrl };
  } catch (e) {
    return {
      ok: false,
      error: e instanceof Error ? e.message : "Checkout failed",
    };
  }
}
```

Calling a Server Action from a Client Component:

```
"use client";
import { beginCheckout } from "./actions";

function CheckoutButton({ ids }: { ids: string[] }) {
  return (
    <button onClick={async () => {
      const result = await beginCheckout("user@example.com", ids);
      if (result.ok && result.redirectUrl) {
        window.location.href = result.redirectUrl;
      }
    }}>Checkout</button>
  );
}
```

8.4 useActionState: The Form Pattern

Wallora uses `useActionState` for form handling:

```
"use client";
import { useActionState } from "react";

const [state, formAction, pending] = useActionState<AuthState, FormData>(action, null);

// In the JSX:
<form action={formAction}>
  <input name="email" type="email" required />
  <button type="submit" disabled={pending}>
    {pending ? "Submitting..." : "Sign Up"}
  </button>
  {state?.error && <p className="text-danger">{state.error}</p>}
</form>
```

8.5 Revalidation & Redirect

After a Server Action modifies data, you can revalidate cached pages:

```
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";

"use server";
export async function saveWallpaper(formData: FormData): Promise<void> {
  await requireAdmin();
  // ... process form data, upsert to database

  revalidatePath("/admin-dash/wallpapers");  // Clear cached pages
  revalidatePath("/wallpapers");
  redirect("/admin-dash/wallpapers");  // Navigate after success
}
```

8.6 `server-only` Package

Sensitive code uses the `server-only` package to prevent accidental client imports:

```
import "server-only";

// This code will throw an error if imported on the client
export const API_SECRET = process.env.PESAPAL_CONSUMER_SECRET;
```

8.7 Exercise: Create a Server Action

Write a server action that adds a wallpaper to a user's favorites:

```
"use server";
import { z } from "zod";
import { revalidatePath } from "next/cache";

const schema = z.object({
  wallpaperId: z.string().min(1),
});

export async function addFavorite(prev: FavoriteState | null, formData: FormData) {
  const parsed = schema.safeParse({ wallpaperId: formData.get("wallpaperId") });
  if (!parsed.success) return { error: "Invalid wallpaper" };

  try {
    // Save to database here
    revalidatePath("/favorites");
    return { success: true };
  } catch (e) {
    return { error: "Failed to add favorite" };
  }
}
```

Chapter 8 Key Points

- Server Components run on the server, can `await` data directly, send only HTML
- Client components (`"use client"`) add interactivity with state and effects
- Server Actions (`"use server"`) run backend logic called from client components
- `useActionState` manages form state and pending status
- `revalidatePath` / `redirect` update UI after mutations
- `server-only` prevents sensitive code from reaching the client

9. SQL & PostgreSQL — Talking to Databases

SQL (Structured Query Language) is how we communicate with databases. Wallora uses PostgreSQL via Supabase. This chapter teaches SQL from scratch, using Wallora's actual database schema as examples.

9.1 What Is SQL?

SQL is pronounced "sequel" or "S-Q-L." It's a language for managing data in relational databases. With SQL you can create tables, insert data, query it, update it, and delete it.

9.2 Creating Tables (DDL)

DDL (Data Definition Language) creates and modifies database structure:

```
-- Wallora's categories table
CREATE TABLE categories (
  id          TEXT PRIMARY KEY,
  slug        TEXT UNIQUE NOT NULL,
  name        TEXT NOT NULL,
  description TEXT NOT NULL DEFAULT ''
);

-- Wallora's wallpapers table (core columns)
CREATE TABLE wallpapers (
  id                    TEXT PRIMARY KEY,
  slug                  TEXT UNIQUE NOT NULL,
  title                 TEXT NOT NULL,
  description           TEXT NOT NULL DEFAULT '',
  price_cents           INTEGER NOT NULL DEFAULT 0,
  is_premium            BOOLEAN NOT NULL DEFAULT false,
  category_slug         TEXT NOT NULL REFERENCES categories(slug),
  tags                  TEXT[] NOT NULL DEFAULT '{}',
  device                device_type NOT NULL,
  width                 INTEGER NOT NULL,
  height                INTEGER NOT NULL,
  downloads             INTEGER NOT NULL DEFAULT 0,
  created_at            TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

9.3 Enums in PostgreSQL

```
CREATE TYPE device_type AS ENUM ('desktop', 'phone', 'tablet');
CREATE TYPE order_status AS ENUM ('pending', 'paid', 'failed', 'cancelled');
```

9.4 SELECT: Querying Data

The most common SQL operation:

```sql
-- Get all wallpapers
SELECT * FROM wallpapers;

-- Get only title and price
SELECT title, price_cents FROM wallpapers;

-- Filter with WHERE
SELECT * FROM wallpapers
WHERE is_premium = true
  AND device = 'phone';

-- Sort results
SELECT * FROM wallpapers
ORDER BY downloads DESC
LIMIT 10;

-- Count matching rows
SELECT COUNT(*) AS total FROM wallpapers
WHERE category_slug = 'nature';

-- Search with LIKE
SELECT * FROM wallpapers
WHERE title ILIKE '%sunset%';  -- Case-insensitive search
```

9.5 INSERT: Adding Data

```sql
INSERT INTO categories (id, slug, name, description)
VALUES ('cat_001', 'nature', 'Nature', 'Beautiful landscapes');

INSERT INTO wallpapers (id, slug, title, price_cents, is_premium, category_slug, device, width, height)
VALUES (
  'wp_001', 'mountain-sunset', 'Mountain Sunset',
  499, true, 'nature', 'desktop', 3840, 2160
);
```

9.6 UPDATE: Modifying Data

```sql
-- Update a wallpaper's price
UPDATE wallpapers
SET price_cents = 299, is_premium = true
WHERE id = 'wp_001';

-- Increment download count (used in Wallora)
UPDATE wallpapers
SET downloads = downloads + 1
WHERE id = 'wp_001';
```

9.7 DELETE: Removing Data

```
DELETE FROM wallpapers WHERE id = 'wp_001';
```

9.8 JOINs: Combining Tables

```
-- Get wallpapers WITH their category names
SELECT w.title, c.name AS category_name
FROM wallpapers w
JOIN categories c ON w.category_slug = c.slug;

-- Left join (keep wallpapers even without a matching category)
SELECT w.title, o.status
FROM wallpapers w
LEFT JOIN orders o ON w.id = ANY(o.items);
```

9.9 Indexes for Performance

Indexes speed up queries on specific columns:

```
CREATE INDEX idx_wallpapers_category ON wallpapers(category_slug);
CREATE INDEX idx_wallpapers_tags ON wallpapers USING GIN(tags);
CREATE INDEX idx_orders_user ON orders(user_id);
```

9.10 Functions & RPC

Wallora uses a PostgreSQL function for safe download counting:

```
CREATE FUNCTION increment_downloads(wp_id TEXT)
RETURNS VOID
LANGUAGE sql
AS $$
  UPDATE wallpapers SET downloads = downloads + 1
  WHERE id = wp_id;
$$;
```

9.11 JSON Queries

PostgreSQL has excellent JSON support. Wallora stores order items as JSONB:

```sql
-- Query JSON data inside orders
SELECT * FROM orders
WHERE items @> '[{"wallpaperId": "wp_001"}]';

-- Extract specific JSON field
SELECT id, items->0->>'title' AS first_item_title
FROM orders;
```

9.12 Exercise: Write Queries

Write SQL to find the top 5 most-downloaded premium wallpapers in the “nature” category:

```sql
SELECT title, downloads, price_cents
FROM wallpapers
WHERE is_premium = true
  AND category_slug = 'nature'
ORDER BY downloads DESC
LIMIT 5;
```

Chapter 9 Key Points

- SQL manages data in relational databases: `CREATE`, `SELECT`, `INSERT`, `UPDATE`, `DELETE`
- `WHERE` filters rows, `ORDER BY` sorts, `LIMIT` restricts results
- JOINs combine data from multiple tables
- Indexes make queries faster on large tables
- PostgreSQL supports JSON, enums, arrays, and custom functions

10. Supabase — Database, Auth & Storage

Supabase is an open-source Firebase alternative. It provides a PostgreSQL database, authentication, file storage, and real-time subscriptions. Wallora uses all three.

10.1 What Is Supabase?

Supabase gives you a full backend without writing server code. It provides:

- **PostgreSQL Database** — with a web UI and REST API
- **Authentication** — email/password, OAuth, magic links
- **Storage** — file upload for premium images
- **Row-Level Security (RLS)** — per-row database permissions
- **Real-time** — live updates via WebSockets

10.2 Supabase Clients in Wallora

Wallora has three types of Supabase clients:

```
// 1. Browser client (for client components)
// src/lib/supabase/client.ts
import { createBrowserClient } from "@supabase/ssr";

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
}

// 2. Server client (for server components/actions)
// src/lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();
  return createServerClient(url!, key!, {
    cookies: { getAll() { return cookieStore.getAll(); } },
  });
}

// 3. Admin client (service role, bypasses RLS)
// src/lib/supabase/admin.ts
import { createClient } from "@supabase/supabase-js";

export const adminClient = createClient(url!, serviceRoleKey!, {
  auth: { persistSession: false },
});
```

10.3 Row-Level Security (RLS)

RLS policies control who can read/write each row. Wallora uses them extensively:

```
-- Anyone can view categories
CREATE POLICY “Public read access”
  ON categories FOR SELECT
  USING (true);

-- Users can only see their own orders
CREATE POLICY “Users view own orders”
  ON orders FOR SELECT
  USING (auth.uid() = user_id);

-- Admins can do everything
CREATE POLICY “Admin full access”
  ON wallpapers
  USING (is_admin());
```

10.4 Triggers & Functions

Wallora auto-creates a profile when a user signs up:

```
CREATE FUNCTION public.handle_new_user()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  INSERT INTO public.profiles (id, email, display_name)
  VALUES (
    NEW.id,
    COALESCE(NEW.email, ''),
    COALESCE(
      NULLIF(NEW.raw_user_meta_data->>'display_name', ''),
      SPLIT_PART(COALESCE(NEW.email, 'user'), '@', 1)
    )
  )
  ON CONFLICT (id) DO NOTHING;
  RETURN NEW;
EXCEPTION WHEN others THEN
  RAISE WARNING 'handle_new_user failed for %: %', NEW.id, SQLERRM;
  RETURN NEW;
END;
$$;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW
  EXECUTE FUNCTION public.handle_new_user();
```

10.5 Supabase Storage

Wallora stores premium wallpaper originals in a private Supabase bucket. Signed URLs provide temporary access:

```
// Generate a signed URL (valid for 60 seconds)
const { data } = await adminClient.storage
  .from("premium-wallpapers")
  .createSignedUrl(filePath, 60);

// Download URL after purchase
return redirect(data?.signedUrl ?? "/");
```

10.6 Feature Flags for Supabase

Wallora works without Supabase (demo mode). Feature flags make it optional:

```
export const features = {
  supabase: Boolean(env.supabaseUrl && env.supabaseAnonKey),
};

export async function getRepo(): Promise<Repository> {
  if (features.supabase) {
    const { supabaseRepo } = await import("./supabase-repo");
    return supabaseRepo;
  }
  return mockRepo;  // In-memory fallback for development
}
```

Chapter 10 Key Points

- Supabase provides PostgreSQL database, auth, and file storage
- Three client types: browser, server, admin (service role)
- Row-Level Security (RLS) controls per-row access
- Triggers automate actions (like creating profiles on signup)
- Signed URLs provide time-limited access to private storage
- Feature flags make Supabase optional for local development

11. Zod Validation — Keeping Data Clean

Zod is a TypeScript-first validation library. It ensures that the data coming into your app (from forms, APIs, or databases) has the correct shape and values. Wallora uses Zod everywhere.

11.1 Why Validate?

Without validation, your app is vulnerable to bad data, crashes, and security issues. Zod provides a single source of truth: your schema defines both runtime validation AND TypeScript types.

11.2 Basic Schemas

```
import { z } from "zod";

// Simple string validation
const emailSchema = z.string().email();
emailSchema.parse("not-an-email");  // Throws error!
emailSchema.parse("user@example.com");  // OK

// Safe parse (no throw, returns result object)
const result = emailSchema.safeParse("bad");
// { success: false, error: ZodError }
```

11.3 Object Schemas

```
// From Wallora's checkout action
const checkoutSchema = z.object({
  email: z.string().email("Please enter a valid email"),
  ids: z.array(z.string().min(1)).min(1, "At least one item required"),
});

const signupSchema = z.object({
  displayName: z.string().min(2, "Name must be at least 2 characters").max(60),
  email: z.string().email(),
  password: z.string().min(8, "Password must be at least 8 characters"),
  dateOfBirth: z.string().min(1),
});

// Use safeParse to validate input
const parsed = signupSchema.safeParse({ displayName, email, password, dateOfBirth });
if (!parsed.success) {
  return { error: parsed.error.issues[0]?.message };
}
```

11.4 Complex Validations

```
// Enums (matching PostgreSQL enums)
const DeviceType = z.enum(["desktop", "phone", "tablet"]);
const AgeRating = z.enum(["everyone", "13+", "16+", "18+"]);

// Nullable fields
const VideoPublicId = z.string().nullable();

// Optional fields
const wallpaperSchema = z.object({
  title: z.string().min(1),
  description: z.string().default(""),  // Default when missing
  tags: z.array(z.string()).default([]),
  videoPublicId: z.string().nullable().optional(),
});

// Advanced: refine with custom validation
const ageSchema = z.number().refine(val => val >= 0 && val < 150, {
  message: "Age must be between 0 and 150",
});
```

11.5 Type Inference with `z.infer`

This is the killer feature: Zod types automatically become TypeScript types:

```
const wallpaperSchema = z.object({
  id: z.string(),
  title: z.string().min(1),
  priceCents: z.number().nonnegative(),
  isPremium: z.boolean(),
  tags: z.array(z.string()).default([]),
});

// Extract TypeScript type from the schema (ONE source of truth)
type WallpaperInput = z.infer<typeof wallpaperSchema>;
// → { id: string; title: string; priceCents: number; isPremium: boolean; tags: string[] }

// Use it in your code:
function saveWallpaper(data: WallpaperInput) { ... }
```

11.6 Zod in Server Actions (Wallora Pattern)

```
"use server";
import { z } from "zod";

const schema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
});

type AuthState = { error?: string } | null;

export async function login(_prev: AuthState, formData: FormData): Promise<AuthState> {
  const parsed = schema.safeParse({
    email: formData.get("email"),
    password: formData.get("password"),
  });

  if (!parsed.success) {
    return { error: parsed.error.issues[0]?.message ?? "Invalid input" };
  }

  // parsed.data is fully typed: { email: string; password: string }
  const { email, password } = parsed.data;

  // ... authenticate with Supabase
}
```

11.7 Zod for AI Output (Gemini)

Wallora even validates AI output with Zod:

```
const analysisSchema = z.object({
  title: z.string().min(1),
  categorySlug: z.string(),
  ageRating: z.enum(["everyone", "13+", "16+", "18+"]),
  tags: z.array(z.string()),
  description: z.string(),
});

type WallpaperAnalysis = z.infer<typeof analysisSchema>;

function validate(raw: unknown): WallpaperAnalysis {
  return analysisSchema.parse(raw);
}
```

11.8 Exercise: Create a Validation Schema

Create a Zod schema for a blog post form:

```
import { z } from "zod";

const postSchema = z.object({
  title: z.string().min(5, "Title must be at least 5 characters"),
  slug: z.string().regex(/^[a-z0-9-]+$/, "Slug must be lowercase with hyphens"),
  excerpt: z.string().max(200).default(""),
  body: z.string().min(1),
  published: z.boolean().default(false),
  tags: z.array(z.string()).default([]),
});

type PostInput = z.infer<typeof postSchema>;
```

Chapter 11 Key Points

- Zod validates data at runtime, catching bad input before it breaks your app
- `safeParse` returns a result object instead of throwing
- `z.infer` extracts TypeScript types from schemas (single source of truth)
- Zod supports strings, numbers, booleans, arrays, enums, nullable, optional, and custom refinements
- Used in Server Actions for form validation, API input, and even AI output

12. Authentication & Security

Authentication verifies who users are. Authorization controls what they can do. Wallora implements both with Supabase Auth and custom access control.

12.1 Authentication vs Authorization

Authentication (“AuthN”) is about identity: “Who are you?”
Authorization (“AuthZ”) is about permissions: “What are you allowed to do?”

Example: signing in is authentication; checking if someone is an admin is authorization.

12.2 Supabase Auth

Supabase handles the heavy lifting of auth: password hashing, session management, email confirmations, and OAuth.

```
// Sign up
const { data, error } = await supabase.auth.signUp({
  email: "user@example.com",
  password: "securepassword",
  options: {
    data: { display_name: "James", date_of_birth: "2000-01-01" },
  },
});

// Sign in
const { data, error } = await supabase.auth.signInWithPassword({
  email: "user@example.com",
  password: "securepassword",
});

// Sign out
await supabase.auth.signOut();
```

12.3 Demo Auth

When Supabase is not configured, Wallora uses cookie-based demo auth:

```ts
// src/lib/demo-auth.ts
interface DemoProfile {
  id: string;
  email: string;
  displayName: string;
  role: "user" | "admin";
}

export function getDemoUser(request: NextRequest): DemoProfile | null {
  const cookie = request.cookies.get("aurava_demo_user");
  if (!cookie) return null;
  try {
    return JSON.parse(cookie.value);
  } catch {
    return null;
  }
}
```

12.4 The `getViewer` Pattern

Wallora uses a unified `getViewer` function that works with both Supabase and demo auth:

```ts
export interface Viewer {
  profile: Profile | null;
  isAdmin: boolean;
  supabaseUser: User | null;
}

export async function getViewer(): Promise<Viewer> {
  if (features.supabase) {
    const supabase = await createServerClient();
    const { data: { user } } = await supabase.auth.getUser();
    if (!user) return { profile: null, isAdmin: false, supabaseUser: null };
    const { data: profile } = await supabase.from("profiles").select("*").eq("id", user.id).single();
    return { profile, isAdmin: profile?.role === "admin", supabaseUser: user };
  }
  // Demo mode: check cookie
  const demo = getDemoUser();
  return { profile: demo, isAdmin: demo?.role === "admin", supabaseUser: null };
}
```

12.5 Authorization: requireAdmin & requireUser

```ts
export async function requireAdmin(): Promise<Viewer> {
  const viewer = await getViewer();
  if (!viewer.isAdmin) {
    redirect("/login");
  }
  return viewer;
}

export async function requireUser(): Promise<Viewer> {
  const viewer = await getViewer();
  if (!viewer.profile) {
    redirect(`/login?next=${encodeURIComponent(currentPath)}`);
  }
  return viewer;
}
```

12.6 Security Best Practices

1. **Never trust client input** — always validate with Zod on the server
2. **Use HTTPS** — encrypts data in transit (Vercel does this automatically)
3. **Hash passwords** — Supabase handles this; never store raw passwords
4. **RLS policies** — database-level security as a safety net
5. **Server-only secrets** — use `server-only` package to prevent client access
6. **Signed URLs for downloads** — time-limited access to protected files
7. **Cron auth** — shared secret for automated tasks

```ts
// Cron authentication with shared secret
export async function verifyCronAuth(request: NextRequest): Promise<boolean> {
  const authHeader = request.headers.get("authorization");
  return authHeader === `Bearer ${process.env.CRON_SECRET}`;
}
```

Chapter 12 Key Points

- Authentication verifies identity; authorization controls access
- Supabase Auth handles signup, login, session management
- Demo auth (cookies) provides fallback when Supabase is unavailable
- `getViewer` unifies both auth methods
- `requireAdmin` and `requireUser` protect routes
- Always validate input, hash passwords, use RLS, keep secrets server-only

13. Payments with PesaPal

Wallora integrates PesaPal v3 for payment processing. PesaPal supports M-Pesa (mobile money), credit cards, and other African payment methods. This chapter walks through the complete payment flow.

13.1 PesaPal Overview

PesaPal is a Kenyan payment gateway. The flow is:

1. **Submit Order** → Send order details to PesaPal
2. **Redirect** → User pays on PesaPal's hosted page (iframe)
3. **IPN (Instant Payment Notification)** → PesaPal pings your server with the result
4. **Callback** → User returns to your site after payment
5. **Fulfill** → Mark order paid, deliver the product

13.2 Getting an Auth Token

```
import "server-only";

async function getToken(): Promise<string> {
  const res = await fetch(`${baseUrl}/api/Auth/RequestToken`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      consumer_key: env.pesapalConsumerKey,
      consumer_secret: env.pesapalConsumerSecret,
    }),
  });

  const data = await res.json();
  return data.token;
}
```

13.3 Submitting an Order

```
export async function submitOrder(input: SubmitOrderInput): Promise<SubmitOrderResult> {
  // Mock mode fallback (when PesaPal isn't configured)
  if (!features.pesapal) {
    return {
      orderTrackingId: `mock_${input.merchantRef}`,
      redirectUrl: abs(`/checkout/mock?ref=${input.merchantRef}`),
    };
  }

  const token = await getToken();
  const res = await fetch(`${baseUrl}/api/Transactions/SubmitOrderRequest`, {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${token}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      id: input.merchantRef,
      currency: "USD",
      amount: input.amount,
      description: "Wallora Wallpaper Purchase",
      callback_url: abs("/checkout/callback"),
      notification_id: env.pesapalIpnId,
      billing_address: { email_address: input.email },
    }),
  });

  const data = await res.json();
  return {
    orderTrackingId: data.order_tracking_id,
    redirectUrl: data.redirect_url,
  };
}
```

13.4 IPN: Receiving Payment Notifications

```
// src/app/api/pesapal/ipn/route.ts
export async function GET(request: NextRequest) {
  const trackingId = request.nextUrl.searchParams.get("pesapal_transaction_tracking_id");
  const merchantRef = request.nextUrl.searchParams.get("pesapal_merchant_reference");

  if (!trackingId || !merchantRef) {
    return NextResponse.json({ error: "Missing params" }, { status: 400 });
  }

  const order = await fulfillByMerchantRef(merchantRef);
  return NextResponse.json(order);
}

export const POST = GET;  // Accept both GET and POST
```

13.5 Fulfillment: Mark Paid & Deliver

```ts
export async function fulfillByMerchantRef(ref: string): Promise<Order | null> {
  const order = await repo.getOrderByMerchantRef(ref);
  if (order?.status === "paid") return order;  // Already fulfilled

  const status = await getTransactionStatus(trackingId);
  if (status !== "completed") {
    await repo.updateOrder(order.id, { status: "failed" });
    return null;
  }

  const paid = await repo.updateOrder(order.id, {
    status: "paid",
    paidAt: new Date().toISOString(),
  });

  // Fire-and-forget delivery (don't block the response)
  after(() => deliverOrder(fulfilled));

  return paid;
}
```

13.6 The Mock Payment Mode

For development, Wallora has a mock payment page that simulates the PesaPal flow:

```tsx
// src/app/checkout/mock/page.tsx
"use client";
export default function MockPaymentPage() {
  const searchParams = useSearchParams();
  const ref = searchParams.get("ref");

  const simulatePayment = async () => {
    await fetch(`/api/pesapal/ipn?pesapal_merchant_reference=${ref}&pesapal_transaction_tracking_id=mock_t
    window.location.href = `/checkout/callback?ref=${ref}`;
  };

  return <button onClick={simulatePayment}>Simulate Payment</button>;
}
```

Chapter 13 Key Points

- PesaPal processes M-Pesa, credit card, and other payments in Africa
- Flow: auth token → submit order → user pays → IPN webhook → fulfill
- IPN (Instant Payment Notification) is the webhook from PesaPal
- Mock mode enables development without real payments
- All payment code is server-only to protect API keys
- Fire-and-forget (`after()`) handles delivery without blocking responses

14. Email & Notifications with Resend

After a successful purchase, Wallora sends a receipt email with download links. This is handled by Resend, a modern email API for developers.

14.1 Setting Up Resend

```
import { Resend } from "resend";

const resend = new Resend(process.env.RESEND_API_KEY);
```

14.2 Sending a Transactional Email

```
export async function sendReceipt(order: Order, downloadUrls: Record<string, string>) {
  const itemsHtml = order.items.map((item) => `
    <tr>
      <td>${item.title}</td>
      <td><a href="${downloadUrls[item.wallpaperId]}">Download</a></td>
    </tr>
  `).join("");

  await resend.emails.send({
    from: `Wallora <${process.env.EMAIL_FROM}>`,
    to: order.email,
    subject: "Your Wallora order receipt",
    html: `
      <h1>Thank you for your order!</h1>
      <p>Your wallpapers are ready to download:</p>
      <table>${itemsHtml}</table>
      <p>Links expire in 60 seconds.</p>
    `,
  });
}
```

14.3 Email with React (JSX Emails)

Resend also supports email templates written in React/JSX:

```
import { Resend } from "resend";
import ReceiptEmail from "@/emails/receipt";  // React component

await resend.emails.send({
  from: "Wallora <noreply@auravaw.tech>",
  to: order.email,
  subject: "Your Wallora order",
  react: <ReceiptEmail order={order} downloadUrls={downloadUrls} />,
});
```

14.4 Feature Flag

Like all services, email is optional:

```
if (!features.resend) {
  console.log("Email not configured: skipping receipt");
  return;
}
```

Chapter 14 Key Points

- Resend is a modern email API for transactional emails
- Send emails with HTML templates or React components
- Feature flags make email optional for development
- Receipt emails include time-limited download links

15. Image Handling with Cloudinary

Cloudinary handles image upload, storage, transformation, optimization, and delivery for Wallora. It's a cloud-based image CDN with powerful URL-based transformations.

15.1 Why Cloudinary?

Self-hosting images is hard: you need storage, resizing, optimization, CDN delivery, and format conversion. Cloudinary gives all of that via simple URL parameters.

```
// A Cloudinary URL with transformations
// https://res.cloudinary.com/demo/image/upload/w_600,q_auto,f_auto/sample.jpg
//                    cloud   resource  transforms   public_id
```

15.2 The Cloudinary URL Builder

```
// src/lib/cloudinary.ts
import { env } from "./env";

function deliveryUrl(resource: string, delivery: string, transforms: string[], id: string): string {
  const base = `https://res.cloudinary.com/${env.cloudinaryCloud}/${resource}/${delivery}`;
  return [base, signature(transforms, id), ...transforms, id].filter(Boolean).join("/");
}

export function previewUrl(w: { previewPublicId: string }, opts: { width?: number } = {}): string {
  const transforms = [
    opts.width ? `w_${opts.width}` : "",
    "c_limit",
    "q_auto",
    "f_auto",
  ].filter(Boolean);

  return deliveryUrl("image", "upload", transforms, w.previewPublicId);
}
```

15.3 Image Transformations

Cloudinary transforms are specified in the URL:

```
// Resize to 600px width (maintains aspect ratio)
w_600,c_limit

// Auto quality (best quality/size ratio)
q_auto

// Auto format (WebP for Chrome, AVIF for Firefox, etc.)
f_auto

// Force download (Content-Disposition: attachment)
fl_attachment

// Combined example
https://res.cloudinary.com/demo/image/upload/w_600,c_limit,q_auto,f_auto/sample.jpg
```

15.4 Signed URLs for Security

To prevent hotlinking, Wallora signs its Cloudinary URLs:

```
function signature(transforms: string[], id: string): string {
  if (!env.cloudinaryApiSecret) return "";

  const toSign = [...transforms, id].filter(Boolean).join("/");
  const digest = createHash("sha1")
    .update(toSign + env.cloudinaryApiSecret)
    .digest("base64");

  return `s--${digest.slice(0, 8).replace(/\//g, "_").replace(/\+/g, "-")}--`;
}
```

15.5 Delivery Strategy

Content Type	Storage	Delivery
Free wallpaper previews	Cloudinary (public)	Optimized URL with transformations
Free wallpaper downloads	Cloudinary	`fl_attachment` forces browser save
Premium wallpaper previews	Cloudinary (resolution-capped to 1600px)	Optimized URL
Premium wallpaper originals	Supabase Storage (private bucket)	60-second signed URL after purchase
Live wallpaper videos	Cloudinary (video)	Looped, capped at 6 seconds

Chapter 15 Key Points

- Cloudinary provides image upload, storage, optimization, and CDN delivery
- Transformations via URL parameters: resize, quality, format, etc.
- `q_auto` + `f_auto` deliver the best quality/size ratio for each browser
- Signed URLs prevent hotlinking and unauthorized access
- Premium originals use Supabase Storage for private, controlled access

16. AI Integration with Google Gemini

Wallora uses Google Gemini to automatically generate metadata (title, description, tags, category) when admins upload new wallpapers. This saves hours of manual work.

16.1 Setting Up Gemini

```
const GEMINI_ENDPOINT = "https://generativelanguage.googleapis.com/v1beta/models";

export function getGeminiModel() {
  if (!features.gemini) return null;
  return env.geminiModel ?? "gemini-2.5-flash";
}
```

16.2 Structured Output with Gemini

Gemini can return structured JSON responses. Wallora pairs this with Zod for validation:

```
const slugs = ["nature", "city", "abstract", "anime", "space", "minimal"];

const prompt = `You are a metadata writer for Aurava wallpaper marketplace.
Analyze the image and return JSON with:
- title: catchy wallpaper title (max 60 chars)
- categorySlug: one of: ${slugs.join(", ")}
- ageRating: "everyone", "13+", "16+", or "18+"
- tags: array of relevant tags (max 8)
- description: SEO-friendly description (max 160 chars)`;

const responseSchema = {
  type: "OBJECT",
  properties: {
    title: { type: "STRING" },
    categorySlug: { type: "STRING", enum: slugs },
    ageRating: { type: "STRING", enum: ["everyone", "13+", "16+", "18+"] },
    tags: { type: "ARRAY", items: { type: "STRING" } },
    description: { type: "STRING" },
  },
};
```

16.3 Calling Gemini

```
const res = await fetch(`${GEMINI_ENDPOINT}/${env.geminiModel}:generateContent`, {
  method: "POST",
  headers: {
    "x-goog-api-key": env.geminiApiKey,
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    contents: [{
      role: "user",
      parts: [
        { text: prompt },
        { inlineData: { mimeType, data: imageBase64 } },
      ],
    }],
    generationConfig: {
      responseMimeType: "application/json",
      responseSchema,
      temperature: 0.4,   // Low temperature = more predictable
    },
  }),
});

const data = await res.json();
const raw = JSON.parse(data.candidates[0].content.parts[0].text);
```

16.4 Validating with Zod

```
function validate(raw: unknown, slugs: string[]): WallpaperAnalysis {
  return analysisSchema.parse(raw);  // Throws if AI returns bad data
}
```

Chapter 16 Key Points

- Gemini auto-generates metadata from wallpaper images
- Structured output mode ensures JSON responses
- Low temperature (0.4) makes AI output more predictable
- Zod validates AI output, catching hallucinated values
- Feature flags make Gemini optional

17. Project Architecture & Repository Pattern

Well-architected apps are easier to maintain, test, and extend. Wallora uses the **Repository Pattern** to abstract data access, making it possible to swap databases without changing business logic.

17.1 The Repository Pattern

A **repository** is an interface that defines data operations. The rest of your code talks to the repository, not directly to the database:

```
// src/lib/repo/types.ts — The interface
export interface Repository {
  // Wallpapers
  getWallpaper(slug: string): Promise<Wallpaper | null>;
  listWallpapers(query: WallpaperQuery): Promise<Wallpaper[]>;
  countWallpapers(query: WallpaperQuery): Promise<number>;
  upsertWallpaper(w: Wallpaper): Promise<Wallpaper>;

  // Categories
  listCategories(): Promise<Category[]>;
  getCategory(slug: string): Promise<Category | null>;

  // Orders
  getOrder(id: string): Promise<Order | null>;
  getOrderByMerchantRef(ref: string): Promise<Order | null>;
  createOrder(o: Order): Promise<Order>;
  updateOrder(id: string, data: Partial<Order>): Promise<Order>;
}
```

17.2 Multiple Implementations

Wallora has TWO implementations of the repository:

Implementation	Used When	Backend
`supabase-repo.ts`	Production (Supabase configured)	PostgreSQL via Supabase JS client
`mock.ts`	Development / demo (no Supabase)	In-memory JavaScript objects

17.3 Lazy-Loaded Factory

```
// src/lib/repo/index.ts — Factory pattern
import { features } from "@/lib/env";

export async function getRepo(): Promise<Repository> {
  if (features.supabase) {
    // Lazy import — only loads when Supabase is configured
    const { supabaseRepo } = await import("./supabase-repo");
    return supabaseRepo;
  }
  return mockRepo;
}
```

17.4 Snake_case to camelCase Mapping

Databases use `snake_case`; TypeScript uses `camelCase`. The repository handles the mapping:

```
// Database row (snake_case)
type WallpaperRow = {
  original_public_id: string;
  is_premium: boolean;
  price_cents: number;
  category_slug: string;
};

// Domain model (camelCase)
function toWallpaper(r: WallpaperRow): Wallpaper {
  return {
    originalPublicId: r.original_public_id,
    isPremium: r.is_premium,
    priceCents: r.price_cents,
    categorySlug: r.category_slug,
  };
}
```

17.5 Feature Flags

Every external service has a feature flag in `src/lib/env.ts`:

```
export const features = {
  supabase: Boolean(env.supabaseUrl && env.supabaseAnonKey),
  cloudinary: Boolean(env.cloudinaryCloud),
  gemini: Boolean(env.geminiApiKey),
  pesapal: Boolean(env.pesapalConsumerKey && env.pesapalConsumerSecret),
  resend: Boolean(env.resendKey),
};
```

17.6 Directory Structure

```
src/
  // Pages
  app/                    // Next.js App Router pages
    page.tsx              // Homepage
    wallpapers/           // Catalog pages
    checkout/             // Checkout flow
    admin-dash/           // Admin dashboard
    api/                  // API routes

  // Components
  components/
    ui.tsx                // Base UI primitives (Button, Container, etc.)
    navbar.tsx
    footer.tsx
    wallpaper-card.tsx
    cart.tsx              // Cart context + provider
    ...

  // Business logic
  lib/
    types.ts              // Domain types
    constants.ts          // Constants, categories
    env.ts                // Environment + feature flags
    utils.ts              // Utility functions (cn, formatPrice)
    auth.ts               // Auth helpers
    catalog.ts            // Catalog read operations
    orders.ts             // Order/checkout logic
    pesapal.ts            // Payment client
    email.ts              // Email (Resend)
    delivery.ts           // Download delivery
    cloudinary.ts         // Cloudinary URL builder
    gemini.ts             // AI image analysis

    repo/                 // Repository pattern
      types.ts            // Interface
      index.ts            // Factory
      supabase-repo.ts    // Supabase implementation
      mock.ts             // In-memory implementation
      seed.ts             // Sample data

    supabase/             // Supabase clients
      client.ts           // Browser client
      server.ts           // Server client with cookies
      admin.ts            // Service role admin client
```

Chapter 17 Key Points

- Repository pattern abstracts data access behind an interface
- Multiple implementations (Supabase + mock) enable testing
- Lazy imports prevent loading unused code
- Feature flags make every external service optional
- Domain models use camelCase; database rows use snake_case
- Clean separation: pages, components, business logic, data access

18. Building the Catalog & Cart

The catalog and cart are the core of Wallora's user experience. This chapter shows how they work together.

18.1 The Homepage Data Flow

```tsx
// src/app/page.tsx — Server Component
export default async function HomePage() {
  // All data fetched in parallel on the server
  const [featured, week, popular, fresh, live, categories] = await Promise.all([
    getFeaturedForDisplay("day"),
    getFeaturedForDisplay("week"),
    listWallpapers({ sort: "popular", limit: 12 }),
    listWallpapers({ sort: "newest", limit: 8 }),
    listWallpapers({ kind: "live", limit: 8 }),
    listCategories(),
  ]);

  return (
    <Container>
      {featured ? <FeaturedHero ... /> : <FallbackHero />}
      <section>
        <SectionHeading title="Trending"
          action={<Link href="/wallpapers?sort=popular">See all</Link>} />
        <MasonryGrid wallpapers={popular} />
      </section>
      {/* More sections */}
    </Container>
  );
}
```

18.2 The Catalog Filter System

```tsx
// Unified filter interface
interface WallpaperQuery {
  category?: string;
  device?: DeviceType;
  sort?: "newest" | "popular" | "price-asc" | "price-desc";
  limit?: number;
  offset?: number;
  kind?: "image" | "live";
}

// Used by both the page AND the repository
export async function listWallpapers(query: WallpaperQuery = {}): Promise<Wallpaper[]> {
  const repo = await getRepo();
  return repo.listWallpapers(query);
}
```

18.3 Cart State Management

The cart uses React Context with localStorage persistence:

```
// Cart line item
interface CartLine {
  wallpaperId: string;
  title: string;
  priceCents: number;
}

// Cart context value exposed to all components
interface CartContextValue {
  lines: CartLine[];
  add: (w: CardWallpaper) => void;
  remove: (id: string) => void;
  has: (id: string) => boolean;
  totalCents: number;
  count: number;
  clear: () => void;
  hydrated: boolean;
}
```

The cart page shows items with prices, a total, and a checkout button:

```
// In the cart page
const cart = useCart();

return (
  <Container>
    <h1>Shopping Cart</h1>
    {cart.lines.map(line => (
      <div key={line.wallpaperId} className="flex justify-between">
        <span>{line.title}</span>
        <span>{formatPrice(line.priceCents)}</span>
        <button onClick={() => cart.remove(line.wallpaperId)}>Remove</button>
      </div>
    ))}
    <div className="mt-4 text-xl font-bold">
      Total: {formatPrice(cart.totalCents)}
    </div>
    <Link href="/checkout">Proceed to Checkout</Link>
  </Container>
);
```

18.4 Currency Localization

Wallora dynamically adjusts prices based on the user's country:

```tsx
const USD_TO_KES = 129;

export function useCurrency(): CurrencyInfo {
  const { country } = useCurrencyContext();

  return useMemo(() => {
    if (country === "KE") {
      return { code: "KES", locale: "en-KE", rate: 129, country };
    }
    return { code: "USD", locale: "en-US", rate: 1, country };
  }, [country]);
}
```

18.5 Age Gating

```tsx
const AGE_RATING_MIN_YEARS = {
  everyone: 0,
  "13+": 13,
  "16+": 16,
  "18+": 18,
} as const;

export function canView(viewer: Viewer, rating: AgeRating): boolean {
  const required = AGE_RATING_MIN_YEARS[rating];
  if (required === 0) return true;  // Everyone can view

  const age = ageFromDob(viewer.profile?.dateOfBirth ?? null);
  if (age === null) return false;  // No DOB = can't verify

  return age >= required;
}
```

Chapter 18 Key Points

- Server Components fetch all data in parallel with `Promise.all`
- WallpaperQuery provides a unified filter interface for the catalog
- Cart uses Context + localStorage for persistence across sessions
- Currency dynamically adjusts for the user's country
- Age gating works server-side based on the user's date of birth

19. Checkout & Order Fulfillment

The checkout flow connects everything: cart, payment, database, email, and file delivery. This chapter traces the complete path from “Buy” click to “Download” link.

19.1 The Complete Checkout Flow

1. User clicks “Checkout” on cart page
2. User enters email on checkout page
3. `beginCheckout` server action validates input with Zod
4. Creates a pending `Order` in the database
5. Calls PesaPal’s `submitOrder` to get a payment URL
6. Redirects user to PesaPal (or mock page in dev)
7. PesaPal sends IPN webhook with payment result
8. `fulfillByMerchantRef` marks order paid
9. Delivery: bumps download counts, generates signed URLs, sends email
10. User accesses download links from the receipt email

19.2 Creating an Order

```
export async function startCheckout(
  items: { wallpaperId: string }[],
  email: string,
  userId: string | null
): Promise<{ redirectUrl: string }> {
  const repo = await getRepo();
  const merchantRef = `ord_${nanoid()}`;

  const order: Order = {
    id: merchantRef,
    userId,
    email,
    items,
    totalCents, // calculated from item prices
    currency: "USD",
    status: "pending",
    pesapalMerchantRef: merchantRef,
    createdAt: new Date().toISOString(),
  };

  await repo.createOrder(order);

  const { redirectUrl } = await submitOrder({
    merchantRef,
    amount: totalCents / 100,
    email,
  });

  return { redirectUrl };
}
```

19.3 Fulfillment: Deliver After Payment

```
export async function deliverOrder(order: Order) {
  // 1. Increment download counts
  for (const item of order.items) {
    await repo.incrementDownloads(item.wallpaperId);
  }

  // 2. Generate signed download URLs
  const downloadUrls: Record<string, string> = {};
  for (const item of order.items) {
    downloadUrls[item.wallpaperId] = await generateSignedUrl(item.wallpaperId);
  }

  // 3. Send receipt email
  await sendReceipt(order, downloadUrls);
}
```

Chapter 19 Key Points

- Checkout flow: create order → submit to PesaPal → IPN callback → fulfill
- Orders start as “pending” and switch to “paid” after IPN confirmation
- Fulfillment bumps download counts, generates signed URLs, sends email
- Fire-and-forget (`after()`) pattern prevents blocking the IPN response

20. Admin Dashboard & CMS

Wallora has a full admin dashboard for managing wallpapers, categories, blog posts, orders, and featured content. It's protected by admin authorization.

20.1 Admin Layout

```
// src/app/admin-dash/layout.tsx
export default async function AdminLayout({ children }: { children: ReactNode }) {
  await requireAdmin();  // Redirects to /login if not admin
  return <div className="flex">
    <AdminSidebar />
    <main>{children}</main>
  </div>;
}
```

20.2 Server Actions for Admin

```
"use server";

export async function saveWallpaper(formData: FormData): Promise<void> {
  await requireAdmin();
  // Validate, build wallpaper object, upsert
  await repo.upsertWallpaper(wallpaper);
  revalidatePath("/admin-dash/wallpapers");
  revalidatePath("/wallpapers");
  redirect("/admin-dash/wallpapers");
}
```

20.3 Admin Stats Dashboard

```tsx
// src/app/admin-dash/page.tsx
export default async function AdminOverview() {
  await requireAdmin();
  const [wallpapers, orders, categories] = await Promise.all([
    repo.listWallpapers({ limit: 9999 }),
    repo.listOrders(),
    repo.listCategories(),
  ]);

  const paid = orders.filter(o => o.status === "paid");
  const revenue = paid.reduce((sum, o) => sum + o.totalCents, 0);
  const downloads = wallpapers.reduce((sum, w) => sum + w.downloads, 0);

  const stats = [
    { label: "Wallpapers", value: String(wallpapers.length) },
    { label: "Paid orders", value: String(paid.length) },
    { label: "Revenue", value: formatPrice(revenue) },
    { label: "Downloads", value: String(downloads) },
  ];

  return /* render stats cards */;
}
```

20.4 Blog CMS

Wallora includes a full blog system with markdown support:

```sql
// Blog post schema (SQL)
CREATE TABLE posts (
  id              TEXT PRIMARY KEY,
  slug            TEXT UNIQUE NOT NULL,
  title           TEXT NOT NULL,
  excerpt         TEXT NOT NULL DEFAULT '',
  body            TEXT NOT NULL DEFAULT '',
  cover_image     TEXT NOT NULL DEFAULT '',
  published       BOOLEAN NOT NULL DEFAULT false,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Chapter 20 Key Points

- Admin layout wraps all dashboard pages with authorization
- Server Actions with `requireAdmin()` protect every mutation
- Dashboard shows key metrics: wallpapers, orders, revenue, downloads
- Blog CMS supports markdown, cover images, publish/draft states
- Seed data enables demo mode for testing the admin panel

21. Deployment & Operations

Wallora is deployed on Vercel with automated CI/CD, cron jobs, and environment-based feature flags.

21.1 Vercel Deployment

Wallora deploys on Vercel with zero configuration needed. Every git push triggers a build and deploy.

```
// vercel.json
{
  "crons": [
    {
      "path": "/api/cron/wallpaper-of-day",
      "schedule": "0 3 * * *"
    },
    {
      "path": "/api/cron/wallpaper-of-week",
      "schedule": "0 7 * * 1"
    }
  ]
}
```

21.2 Cron Jobs: Featured Wallpaper Rotation

Wallora automatically rotates featured wallpapers daily and weekly:

```
// src/app/api/cron/wallpaper-of-day/route.ts
export async function GET(request: NextRequest) {
  if (!(await verifyCronAuth(request))) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  await refreshWallpaperOfDay();
  return NextResponse.json({ ok: true });
}
```

21.3 Environment Variables

All configuration lives in environment variables. Each service is optional:

```
# .env.local
NEXT_PUBLIC_SITE_URL=https://www.auravaw.tech/
NEXT_PUBLIC_SUPABASE_URL= ...
NEXT_PUBLIC_SUPABASE_ANON_KEY= ...
NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME= ...
GEMINI_API_KEY= ...
PESAPAL_CONSUMER_KEY= ...
RESEND_API_KEY= ...
CRON_SECRET= ...
```

21.4 Operations Checklist

Task	How
Deploy	Push to main branch (Vercel auto-deploys)
Database migrations	Run SQL in Supabase dashboard or via CLI
Featured rotation	Vercel Cron runs daily/weekly
Monitoring	Vercel Analytics, Supabase logs
Backups	Supabase automated backups
Secret rotation	Update env vars in Vercel dashboard

Chapter 21 Key Points

- Vercel deploys automatically from git pushes
- Cron jobs automate featured wallpaper rotation
- Each external service is behind a feature flag
- Environment variables control all configuration
- Never commit secrets; use Vercel’s environment variable dashboard

22. Your Journey Ahead

You've made it through 21 chapters of real-world web development. This final chapter points you toward what to learn next and how to keep building.

22.1 What You've Learned

Let's recap everything this book covered:

Language/Tool	You Can Now
HTML & JSX	Structure web pages with semantic tags; use JSX expressions and conditional rendering
CSS & Tailwind	Style with utility classes; use OKLCH colors, flexbox, grid, masonry, responsive design
JavaScript	Use modern JS features: arrow functions, destructuring, async/await, array methods
TypeScript	Add static types, interfaces, generics, discriminated unions, and Zod type inference
React	Build components with props, state, hooks (useState, useEffect, useCallback, useMemo), context
Next.js	Use App Router, layouts, server components, server actions, API routes, proxy
SQL	Create tables, query with SELECT/WHERE/JOIN, use indexes and functions
Supabase	Set up database, auth, RLS policies, storage with signed URLs
Zod	Validate data and extract TypeScript types from schemas
Payments	Integrate PesaPal with IPN webhooks and mock mode
Email	Send transactional emails with Resend
Image CDN	Use Cloudinary for image transformations, optimization, and delivery
AI	Call Gemini APIs with structured output for auto-metadata

22.2 Your Next Projects

Now that you understand Wallora, here are ideas to build on your own:

- **E-commerce store** — Same architecture, different products (clothing, art, digital goods)
- **Photo gallery** — Replace wallpapers with photography portfolio
- **SaaS dashboard** — Use the admin pattern for analytics, user management
- **Blog + newsletter** — Blog CMS with email subscriptions
- **Digital downloads marketplace** — Templates, fonts, music, software

22.3 Next Steps for Learning

1. **Build a project end-to-end** — The best way to learn is by doing
2. **Read the docs** — Next.js docs, React docs, Supabase docs
3. **Join communities** — GitHub Discussions, Discord servers, Stack Overflow
4. **Contribute to open source** — Find projects on GitHub and submit PRs
5. **Learn testing** — Vitest, Playwright, React Testing Library
6. **Learn databases deeper** — Indexing strategies, query optimization, migrations
7. **Learn DevOps** — Docker, CI/CD pipelines, monitoring

22.4 Final Words

> *"The best way to predict the future is to create it." — Peter Drucker*

Every expert was once a beginner. The code you've seen in this book was written by real developers facing real problems. Now you have the tools to solve those same problems yourself.

Start small. Build something every day. Break things and fix them. Share your work. The web development community is vast and welcoming. You are not alone.

Now go build something amazing.

Chapter 22 Key Points

- You've learned a production-grade tech stack: TypeScript, React, Next.js, SQL, and more
- Build projects to solidify your knowledge
- Join the community and contribute to open source
- Keep learning: testing, databases, DevOps
- Start small, build daily, share your work

— End of Book —

Built with Wallora code as reference. Now go build your own.

Aurava.wallora • 2026